

Task 4 — Web Security Testing

Manually demonstrating 3 vulnerabilities in OWASP Juice Shop

Intern	Prajwal
Internship	Indravex Technologies — Cyber Security Internship
Target	OWASP Juice Shop — http://localhost:3000 (local instance)
Date	6 September 2026

1. Objective

Manually find and demonstrate three real vulnerabilities in a deliberately vulnerable app — not run a scanner and copy its output, but actually trigger each issue by hand and understand why it works.

2. Tools Used

- Browser (Firefox) — manually crafting and submitting payloads
- OWASP Juice Shop — the target application, running locally in Docker

3. Methodology

Picked three vulnerability types from the OWASP-style list — SQL Injection, XSS, and Broken Authentication — and tested each directly through the app's normal UI (login form, search bar), rather than using an automated scanner. For the SQL Injection specifically, this builds on the same finding already confirmed via ZAP in Task 1, but here demonstrated manually to show the actual login-bypass impact rather than just a database error.

4. Steps Performed

SQL Injection — login bypass:

- On the login page, entered ' OR 1=1-- as the email, any value as password
- Logged in successfully without knowing any real password

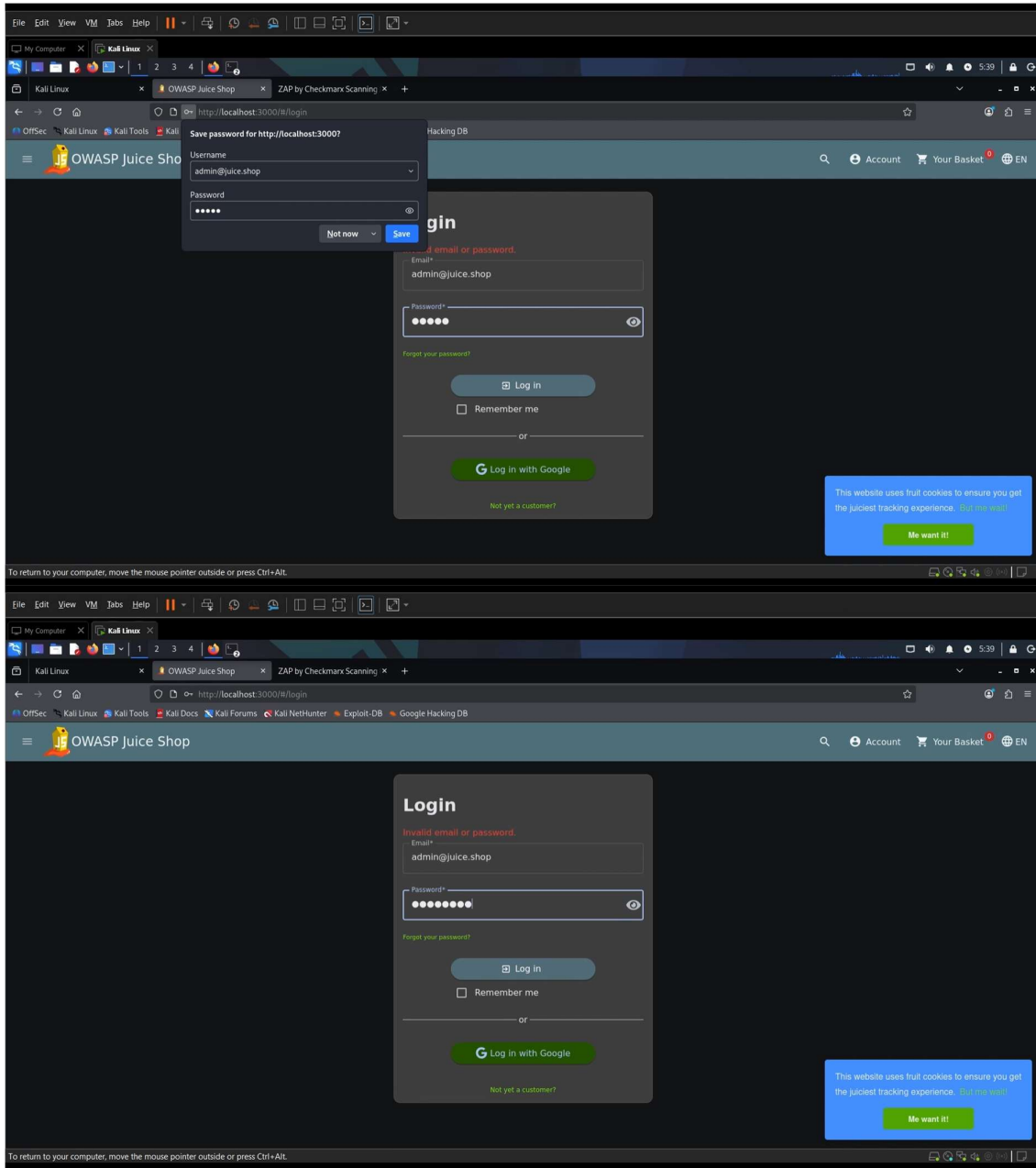
XSS — search bar:

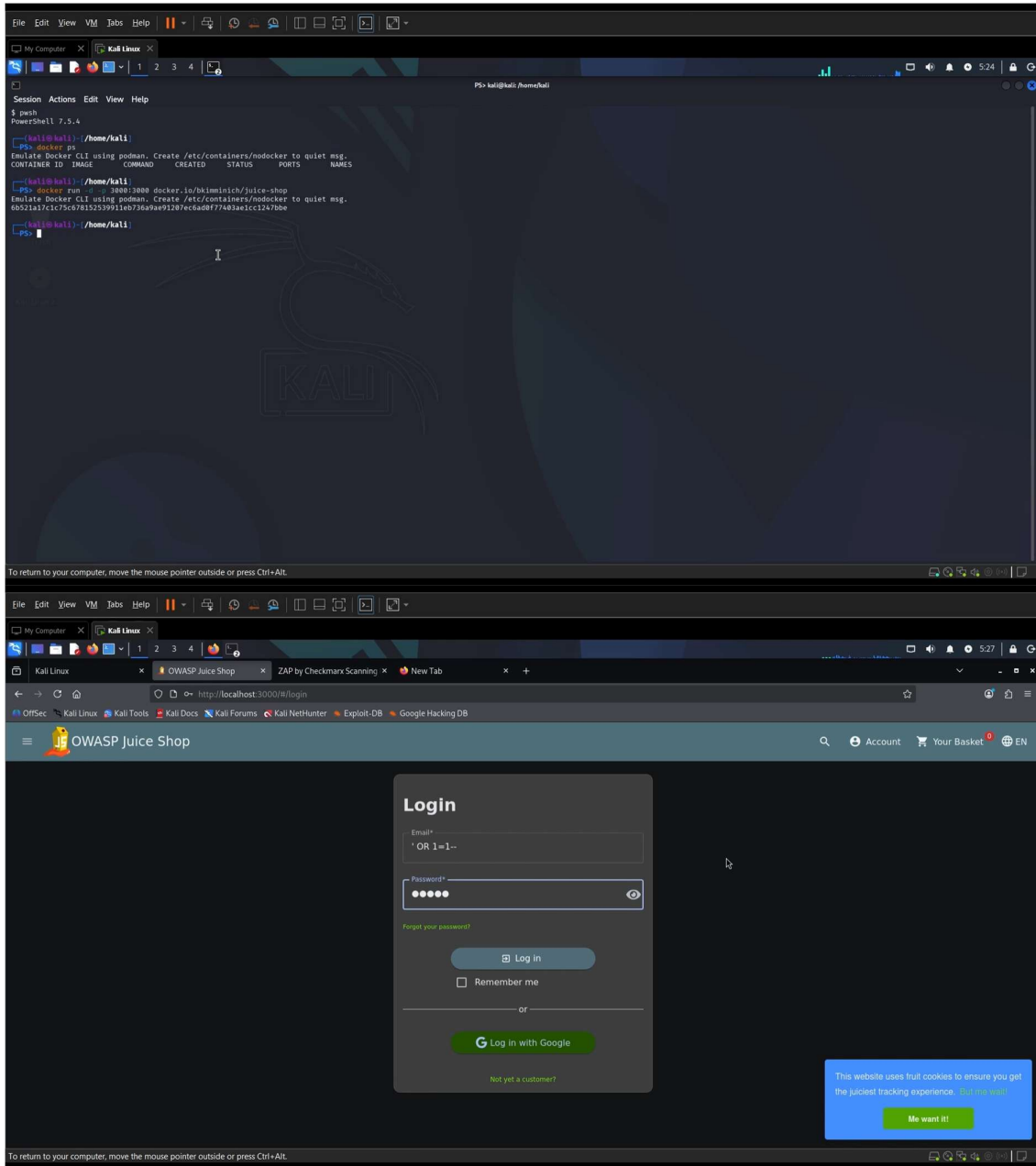
- First tried `<script>alert('XSS')</script>` — blocked by the browser's built-in script-tag protection
- Switched to `<img src=x onerror=alert('XSS')>` — this fired successfully, popping the alert box

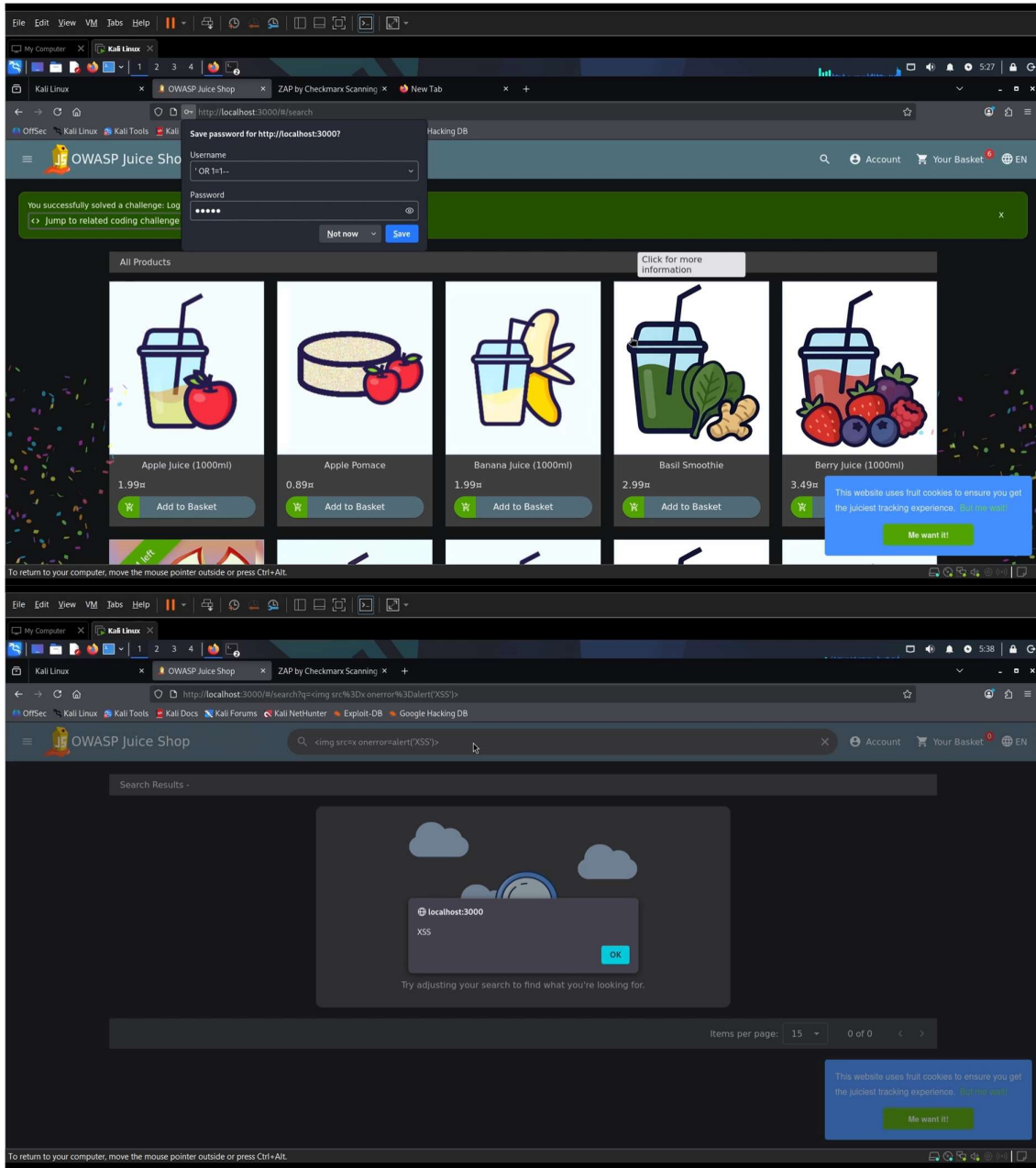
Broken Authentication:

- Attempted several rapid, incorrect logins against a known account (admin@juice-sh.op)
- No lockout, delay, or CAPTCHA appeared after repeated failures

5. Screenshots







6. Findings

Vulnerability	Where / Payload	Impact
SQL Injection	' OR 1=1--	Full login bypass without valid credentials
XSS (DOM-based)		Arbitrary JavaScript execution in the victim's browser
Broken Authentication	Repeated login attempts	No logout — account open to unlimited password guessing

7. Risk Analysis

The SQL Injection is the most serious of the three — it doesn't just leak an error message, it lets you log in as another user (potentially admin) with zero knowledge of their password. That's a complete authentication bypass, not just a data leak.

The XSS finding matters because it proves the app will run attacker-supplied JavaScript in a real user's browser. In a real attack, the `alert()` would instead be something like stealing a session cookie or redirecting the user — the popup is just the safe way to prove the same execution path works.

Broken Authentication on its own isn't catastrophic, but it's what makes the other two worse — no rate limiting means an attacker can brute-force passwords freely, and combined with the SQLi bypass, there's effectively no real barrier to getting into an account.

8. Recommendations

- Use parameterized queries for all login and search functionality — never build SQL from raw string concatenation
- Sanitize and encode all user input before rendering it back to the page, and add a Content-Security-Policy header as a second layer of defense
- Add account logout after a small number of failed attempts, plus rate limiting and CAPTCHA on the login endpoint

9. Conclusion

All three vulnerabilities were confirmed by hand rather than just trusting a scanner's alert list — which mattered most for the XSS test, where the first payload attempt actually failed and had to be adjusted to get past the browser's own script-tag protection. Together, these three findings show a common thread: the app trusts user input by default instead of validating or restricting it, whether that input ends up in a SQL query, rendered on a page, or hammering a login form.